```
lohri_summary = pd.DataFrame(lohri_rows)
print("\n=== LOHRI SUMMARY ===")
if not lohri_summary.empty:
    print(lohri_summary.round({'PM10R2': 3, 'PM2.5MAPE': 1, 'PM10MAPE': 1}))
```

```
=== DIWALI SUMMARY ===
   year   PM2.5MAE  PM2.5R2  PM2.5MAPE    PM10MAE  PM10R2  PM10MAPE
0  2019  16.044674    0.837       17.4  19.994994   0.812      12.8
1  2020  14.390252    0.738       17.5  24.885961   0.778      14.1
2  2021  17.252834    0.783       24.6  26.926584   0.837      15.4
3  2022  14.324854    0.858       22.8  22.716007   0.876      16.6
4  2023  12.926686    0.701       16.2  25.106771   0.738      11.7
5  2024  14.108789    0.333       15.8  33.496101   0.360      15.4
6  2025  18.479174    0.533       39.9  36.499001   0.207      37.7

=== LOHRI SUMMARY ===
   year   PM2.5MAE  PM2.5MAPE    PM10MAE  PM10R2  PM10MAPE
0  2019  13.750896       20.9  21.226868   0.346      15.3
1  2020   8.829342       17.7  19.902739   0.435      15.1
2  2021  14.332743       19.9  22.207308   0.656      16.7
3  2022  14.161713       27.7  25.576763   0.815      19.3
4  2023  16.299434       22.1  22.436835   0.669      16.9
5  2024  16.631942       21.4  30.361829   0.337      15.7
6  2025  16.537610       31.1  30.183710   0.225      22.0
```

NEW PIPELINE STARTS HERE

Double-click (or enter) to edit

```
from google.colab import drive
drive.mount('/content/drive')
base = "/content/drive/MyDrive/PBL/"
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force
```

```
df_police = pd.concat([
    pd.read_csv(f"{base}Police_2019_2024.csv"),
    pd.read_csv(f"{base}Police_2025_clean.csv")
])

df_shastri = pd.concat([
    pd.read_csv(f"{base}Shastri_2019_2024.csv"),
    pd.read_csv(f"{base}Shastri_2025_clean.csv")
])

df_adarsh = pd.concat([
    pd.read_csv(f"{base}Adarsh_Nagar_2019_2024_combined.csv"),
    pd.read_csv(f"{base}Adarsh_2025_clean.csv")
])

df_met = pd.read_csv(
    f"{base}POWER_POINT_HOURLY.csv",
    skiprows=12,   # keep your skiprows
    sep=","        # IMPORTANT FIX
)
```

```
print(df_met.head())
print(df_met.columns)
```

```
   YEAR  MO  DY  HR   T2M   RH2M     PS  WS2M
0  2019   1   1   0  8.96  28.10  97.28  1.43
1  2019   1   1   1  8.23  30.23  97.27  1.45
2  2019   1   1   2  7.61  32.12  97.23  1.47
3  2019   1   1   3  7.14  33.78  97.20  1.49
4  2019   1   1   4  6.81  35.02  97.19  1.48
Index(['YEAR', 'MO', 'DY', 'HR', 'T2M', 'RH2M', 'PS', 'WS2M'], dtype='object')
```

```
df_met["Timestamp"] = pd.to_datetime(
    df_met[["YEAR", "MO", "DY", "HR"]].rename(columns={
        "YEAR": "year",
        "MO": "month",
        "DY": "day",
        "HR": "hour"
    })
)
```

```
df_met = df_met.rename(columns={
    "T2M": "temp",
    "RH2M": "humidity",
    "WS2M": "wind_speed"
})
```

```
df_met = df_met[["Timestamp", "temp", "humidity", "wind_speed"]]
```

```
print(df_met.head())
print(df_met["Timestamp"].min(), df_met["Timestamp"].max())
```

```
            Timestamp  temp  humidity  wind_speed
0 2019-01-01 00:00:00  8.96     28.10        1.43
1 2019-01-01 01:00:00  8.23     30.23        1.45
2 2019-01-01 02:00:00  7.61     32.12        1.47
3 2019-01-01 03:00:00  7.14     33.78        1.49
4 2019-01-01 04:00:00  6.81     35.02        1.48
2019-01-01 00:00:00 2025-12-31 23:00:00
```

```
df_police["Timestamp"] = pd.to_datetime(df_police["Timestamp"])
df_adarsh["Timestamp"] = pd.to_datetime(df_adarsh["Timestamp"])
df_shastri["Timestamp"] = pd.to_datetime(df_shastri["Timestamp"])
print(df_police["Timestamp"].dtype)
print(df_met["Timestamp"].dtype)
```

```
datetime64[ns]
datetime64[ns]
```

```
df_multi = df_police.merge(df_adarsh, on="Timestamp", how="inner") \
                    .merge(df_shastri, on="Timestamp", how="inner") \
                    .merge(df_met, on="Timestamp", how="inner")

df_multi = df_multi.sort_values("Timestamp").reset_index(drop=True)
```

```
print(df_multi.shape)
```

```
(7671, 28)
```

```
full_df_multi = df_multi.copy()
full_df_multi = full_df_multi.interpolate()
```

Double-click (or enter) to edit

```
full_df_multi["year"] = full_df_multi["Timestamp"].dt.year
full_df_multi["month"] = full_df_multi["Timestamp"].dt.month
full_df_multi["day"] = full_df_multi["Timestamp"].dt.day
full_df_multi["hour"] = full_df_multi["Timestamp"].dt.hour
full_df_multi["dayofweek"] = full_df_multi["Timestamp"].dt.dayofweek
```

```
import numpy as np

full_df_multi["hour_sin"] = np.sin(2 * np.pi * full_df_multi["hour"] / 24)
full_df_multi["hour_cos"] = np.cos(2 * np.pi * full_df_multi["hour"] / 24)
```

```
print(df_multi.columns)
```

```
Index(['Timestamp', 'PM2.5 (µg/m³)_x', 'PM10 (µg/m³)_x', 'NO (µg/m³)_x',
       'NO2 (µg/m³)_x', 'NH3 (µg/m³)_x', 'SO2 (µg/m³)_x', 'CO (mg/m³)_x',
       'Ozone (µg/m³)_x', 'PM2.5 (µg/m³)_y', 'PM10 (µg/m³)_y', 'NO (µg/m³)_y',
       'NO2 (µg/m³)_y', 'NH3 (µg/m³)_y', 'SO2 (µg/m³)_y', 'CO (mg/m³)_y',
       'Ozone (µg/m³)_y', 'PM2.5 (µg/m³)', 'PM10 (µg/m³)', 'NO (µg/m³)',
       'NO2 (µg/m³)', 'NH3 (µg/m³)', 'SO2 (µg/m³)', 'CO (mg/m³)',
       'Ozone (µg/m³)', 'temp', 'humidity', 'wind_speed'],
      dtype='object')
```

```
df_multi = df_multi.rename(columns={
    "PM2.5 (µg/m³)_x": "PM2.5_police",
    "PM2.5 (µg/m³)_y": "PM2.5_adarsh",
    "PM2.5 (µg/m³)": "PM2.5_shastri",

    "PM10 (µg/m³)_x": "PM10_police",
    "PM10 (µg/m³)_y": "PM10_adarsh",
    "PM10 (µg/m³)": "PM10_shastri"
})

full_df_multi = df_multi.copy()
```

```
full_df_multi["PM2.5_avg"] = (
    full_df_multi["PM2.5_police"] +
    full_df_multi["PM2.5_adarsh"] +
    full_df_multi["PM2.5_shastri"]
) / 3

full_df_multi["PM2.5_diff_pa"] = full_df_multi["PM2.5_police"] - full_df_multi["PM2.5_adarsh"]
full_df_multi["PM2.5_diff_ps"] = full_df_multi["PM2.5_police"] - full_df_multi["PM2.5_shastri"]
```

```
full_df_multi["PM2.5_police_t-1"] = full_df_multi["PM2.5_police"].shift(1)
full_df_multi["PM2.5_police_t-2"] = full_df_multi["PM2.5_police"].shift(2)

full_df_multi["PM2.5_adarsh_t-1"] = full_df_multi["PM2.5_adarsh"].shift(1)
full_df_multi["PM2.5_shastri_t-1"] = full_df_multi["PM2.5_shastri"].shift(1)

full_df_multi["temp_t-1"] = full_df_multi["temp"].shift(1)
full_df_multi["humidity_t-1"] = full_df_multi["humidity"].shift(1)
```

```
full_df_multi = full_df_multi.dropna()
```

```
feature_cols = [
    "PM2.5_police_t-1", "PM2.5_police_t-2",
    "PM2.5_adarsh_t-1", "PM2.5_shastri_t-1",
    "PM2.5_avg", "PM2.5_diff_pa", "PM2.5_diff_ps",
    "temp", "humidity", "wind_speed",
    "temp_t-1", "humidity_t-1",
    "hour_sin", "hour_cos", "dayofweek", "month"
]

target = "PM2.5_police"
```

```
full_df_multi["year"] = full_df_multi["Timestamp"].dt.year
full_df_multi["month"] = full_df_multi["Timestamp"].dt.month
full_df_multi["day"] = full_df_multi["Timestamp"].dt.day
full_df_multi["hour"] = full_df_multi["Timestamp"].dt.hour
full_df_multi["dayofweek"] = full_df_multi["Timestamp"].dt.dayofweek
```

```
import numpy as np

full_df_multi["hour_sin"] = np.sin(2 * np.pi * full_df_multi["hour"] / 24)
full_df_multi["hour_cos"] = np.cos(2 * np.pi * full_df_multi["hour"] / 24)
print("hour_sin" in full_df_multi.columns)
print("hour_cos" in full_df_multi.columns)
```

```
True
True
```

```
train_mask = full_df_multi["year"] <= 2023
test_mask  = full_df_multi["year"] >= 2024

X_train = full_df_multi.loc[train_mask, feature_cols]
y_train = full_df_multi.loc[train_mask, target]

X_test = full_df_multi.loc[test_mask, feature_cols]
y_test = full_df_multi.loc[test_mask, target]
```

```
print(full_df_multi.columns)
```

```
Index(['Timestamp', 'PM2.5_police', 'PM10_police', 'NO (µg/m³)_x',
       'NO2 (µg/m³)_x', 'NH3 (µg/m³)_x', 'SO2 (µg/m³)_x', 'CO (mg/m³)_x',
       'Ozone (µg/m³)_x', 'PM2.5_adarsh', 'PM10_adarsh', 'NO (µg/m³)_y',
       'NO2 (µg/m³)_y', 'NH3 (µg/m³)_y', 'SO2 (µg/m³)_y', 'CO (mg/m³)_y',
       'Ozone (µg/m³)_y', 'PM2.5_shastri', 'PM10_shastri', 'NO (µg/m³)',
       'NO2 (µg/m³)', 'NH3 (µg/m³)', 'SO2 (µg/m³)', 'CO (mg/m³)',
       'Ozone (µg/m³)', 'temp', 'humidity', 'wind_speed', 'PM2.5_avg',
       'PM2.5_diff_pa', 'PM2.5_diff_ps', 'PM2.5_police_t-1',
       'PM2.5_police_t-2', 'PM2.5_adarsh_t-1', 'PM2.5_shastri_t-1', 'temp_t-1',
       'humidity_t-1', 'year', 'month', 'day', 'hour', 'dayofweek', 'hour_sin',
       'hour_cos'],
      dtype='object')
```

```
# ================================
# 1. SETUP
# ================================
import pandas as pd
import numpy as np

from sklearn.ensemble import RandomForestRegressor
```

```
from sklearn.metrics import r2_score, mean_absolute_error

from google.colab import drive
drive.mount('/content/drive')

base = "/content/drive/MyDrive/PBL/"


# ================================
# 2. LOAD DATA
# ================================
df_police = pd.concat([
    pd.read_csv(f"{base}Police_2019_2024.csv"),
    pd.read_csv(f"{base}Police_2025_clean.csv")
])

df_shastri = pd.concat([
    pd.read_csv(f"{base}Shastri_2019_2024.csv"),
    pd.read_csv(f"{base}Shastri_2025_clean.csv")
])

df_adarsh = pd.concat([
    pd.read_csv(f"{base}Adarsh_Nagar_2019_2024_combined.csv"),
    pd.read_csv(f"{base}Adarsh_2025_clean.csv")
])


# ================================
# 3. CLEAN COLUMN NAMES (IMPORTANT)
# ================================
def clean_columns(df):
    return df.rename(columns={
        "PM2.5 (µg/m³)": "PM2.5",
        "PM10 (µg/m³)": "PM10",
        "NO (µg/m³)": "NO",
        "NO2 (µg/m³)": "NO2",
        "NH3 (µg/m³)": "NH3",
        "SO2 (µg/m³)": "SO2",
        "CO (mg/m³)": "CO",
        "Ozone (µg/m³)": "Ozone"
    })

df_police = clean_columns(df_police)
df_shastri = clean_columns(df_shastri)
df_adarsh = clean_columns(df_adarsh)


# ================================
# 4. ADD STATION + CONCAT
# ================================
df_police["station"] = "police"
df_shastri["station"] = "shastri"
df_adarsh["station"] = "adarsh"

df_all = pd.concat([df_police, df_shastri, df_adarsh], ignore_index=True)


# ================================
# 5. LOAD WEATHER
# ================================
df_met = pd.read_csv(
    f"{base}POWER_POINT_HOURLY.csv",
    skiprows=12
)

df_met["Timestamp"] = pd.to_datetime(
    df_met[["YEAR","MO","DY","HR"]].rename(columns={
        "YEAR":"year","MO":"month","DY":"day","HR":"hour"
    })
)

df_met = df_met.rename(columns={
    "T2M": "temp",
    "RH2M": "humidity",
    "WS2M": "wind_speed"
})

df_met = df_met[["Timestamp","temp","humidity","wind_speed"]]


# ================================
# 6. TIMESTAMP FIX
```

```
# ================================
df_all["Timestamp"] = pd.to_datetime(df_all["Timestamp"])
df_met["Timestamp"] = pd.to_datetime(df_met["Timestamp"])


# ================================
# 7. MERGE WEATHER
# ================================
df = df_all.merge(df_met, on="Timestamp", how="left")


# ================================
# 8. SORT
# ================================
df = df.sort_values(["station","Timestamp"]).reset_index(drop=True)


# ================================
# 9. TIME FEATURES
# ================================
df["year"] = df["Timestamp"].dt.year
df["month"] = df["Timestamp"].dt.month
df["day"] = df["Timestamp"].dt.day
df["hour"] = df["Timestamp"].dt.hour
df["dayofweek"] = df["Timestamp"].dt.dayofweek

df["hour_sin"] = np.sin(2*np.pi*df["hour"]/24)
df["hour_cos" ] = np.cos(2*np.pi*df["hour"]/24)


# ================================
# 10. DIWALI FEATURES (FIXED NAME)
# ================================
from datetime import timedelta

diwali_dates = {
    2019: "2019-10-27",
    2020: "2020-11-14",
    2021: "2021-11-04",
    2022: "2022-10-24",
    2023: "2023-11-12",
    2024: "2024-10-31",
    2025: "2025-10-20",
}

df["is_diwali_period"] = 0
df["days_since_diwali"] = 0

for year, date in diwali_dates.items():
    d = pd.Timestamp(date)
    start = d - timedelta(days=14)
    end   = d + timedelta(days=14)

    mask = (df["Timestamp"] >= start) & (df["Timestamp"] <= end)

    df.loc[mask, "is_diwali_period"] = 1
    df.loc[mask, "days_since_diwali"] = (
        df.loc[mask, "Timestamp"] - d
    ).dt.days

# ================================
# 10b. LOHRI FEATURES (NEW)
# ================================
lohri_dates = {
    2019: "2019-01-13",
    2020: "2020-01-13",
    2021: "2021-01-13",
    2022: "2022-01-13",
    2023: "2023-01-13",
    2024: "2024-01-13",
    2025: "2025-01-13",
}

df["is_lohri_period"] = 0
df["days_since_lohri"] = 0

for year, date in lohri_dates.items():
    d = pd.Timestamp(date)
    start = d - timedelta(weeks=4)
    end = d + timedelta(weeks=4)
    mask = (df["Timestamp"] >= start) & (df["Timestamp"] <= end) & (df["year"] == year)
```

```
    df.loc[mask, "is_lohri_period"] = 1
    df.loc[mask, "days_since_lohri"] = (df.loc[mask, "Timestamp"] - d).dt.days


# ================================
# 11. LAG FEATURES (station-wise)
# ================================
df["PM2.5_t-1"] = df.groupby("station")["PM2.5"].shift(1)
df["PM2.5_t-2"] = df.groupby("station")["PM2.5"].shift(2)
df["PM2.5_t-3"] = df.groupby("station")["PM2.5"].shift(3)
df["PM2.5_t-24"] = df.groupby("station")["PM2.5"].shift(24)

df["PM10_t-1"] = df.groupby("station")["PM10"].shift(1)
df["PM10_t-2"] = df.groupby("station")["PM10"].shift(2)


# ================================
# 12. ROLLING FEATURES
# ================================
df["PM2.5_roll_mean_24"] = (
    df.groupby("station")["PM2.5"]
    .rolling(24)
    .mean()
    .reset_index(level=0, drop=True)
)

df["PM2.5_roll_std_24"] = (
    df.groupby("station")["PM2.5"]
    .rolling(24)
    .std()
    .reset_index(level=0, drop=True)
)

# Also create rolling features for PM10
df["PM10_roll_mean_24"] = (
    df.groupby("station")["PM10"]
    .rolling(24)
    .mean()
    .reset_index(level=0, drop=True)
)
df["PM10_roll_std_24"] = (
    df.groupby("station")["PM10"]
    .rolling(24)
    .std()
    .reset_index(level=0, drop=True)
)


# ================================
# 13. DROP NaNs
# ================================
df = df.dropna()


# ================================
# 14. ENCODE STATION
# ================================
df = pd.get_dummies(df, columns=["station"])


# ================================
# 15. FEATURES
# ================================
feature_cols = [
    "PM2.5_t-1","PM2.5_t-2","PM2.5_t-3","PM2.5_t-24",
    "PM10_t-1","PM10_t-2",
    "PM2.5_roll_mean_24","PM2.5_roll_std_24",
    "PM10_roll_mean_24","PM10_roll_std_24", # New PM10 rolling features
    "temp","humidity","wind_speed",
    "hour_sin","hour_cos","dayofweek","month",
    "is_diwali_period", "days_since_diwali",
    "is_lohri_period", "days_since_lohri",
    "station_police","station_shastri","station_adarsh"
]

target_pm25 = "PM2.5"
target_pm10 = "PM10"


# ================================
# 16. TRAIN / TEST SPLIT
# ================================
```

```
train = df[df["year"] <= 2023]
test  = df[df["year"] >= 2024]

X_train = train[feature_cols]
y_train_pm25 = train[target_pm25]
y_train_pm10 = train[target_pm10]

X_test = test[feature_cols]
y_test_pm25 = test[target_pm25]
y_test_pm10 = test[target_pm10]


# ================================
# 17. SAMPLE WEIGHTS (for PM2.5)
# ================================
weights_pm25 = np.where(y_train_pm25 > 150, 3.0, 1.0)


# ================================
# 18. MODELS (for PM2.5 and PM10)
# ================================
model_pm25 = RandomForestRegressor(
    n_estimators=150,
    max_depth=10,
    random_state=42,
    n_jobs=-1
)
model_pm25.fit(X_train, y_train_pm25, sample_weight=weights_pm25)

model_pm10 = RandomForestRegressor(
    n_estimators=150,
    max_depth=10,
    random_state=42,
    n_jobs=-1
)
model_pm10.fit(X_train, y_train_pm10)


# ================================
# 19. EVALUATION (PM2.5)
# ================================
pred_pm25 = model_pm25.predict(X_test)

mask_diwali = test["is_diwali_period"] == 1

print("PM2.5 Diwali R2:", r2_score(y_test_pm25[mask_diwali], pred_pm25[mask_diwali]))
print("PM2.5 Diwali MAE:", mean_absolute_error(y_test_pm25[mask_diwali], pred_pm25[mask_diwali]))
print("PM2.5 R2:", r2_score(y_test_pm25, pred_pm25))
print("PM2.5 MAE:", mean_absolute_error(y_test_pm25, pred_pm25))

# ================================
# 20. EVALUATION (PM10)
# ================================
pred_pm10 = model_pm10.predict(X_test)

print("\nPM10 Diwali R2:", r2_score(y_test_pm10[mask_diwali], pred_pm10[mask_diwali]))
print("PM10 Diwali MAE:", mean_absolute_error(y_test_pm10[mask_diwali], pred_pm10[mask_diwali]))
print("PM10 R2:", r2_score(y_test_pm10, pred_pm10))
print("PM10 MAE:", mean_absolute_error(y_test_pm10, pred_pm10))
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force
PM2.5 Diwali R2: 0.596501763473962
PM2.5 Diwali MAE: 15.916823929757163
PM2.5 R2: 0.7052705255983786
PM2.5 MAE: 9.642446993679279

PM10 Diwali R2: 0.544975643549461
PM10 Diwali MAE: 29.334252822997048
PM10 R2: 0.5953921183681099
PM10 MAE: 21.902626959705927
```

```
for yr in sorted(test["year"].unique()):
    m = (test["year"] == yr) & (test["is_diwali_period"] == 1)

    if m.sum() == 0:
        continue

    # Use the correct variables (y_test_pm25 and pred_pm25) from the new pipeline
    # And convert the boolean Series 'm' to a numpy array for indexing the prediction array
    print(f"\nYear {yr}")
    print("R2:", r2_score(y_test_pm25[m], pred_pm25[m.values]))
    print("MAE:", mean_absolute_error(y_test_pm25[m], pred_pm25[m.values]))
```

```
Year 2024
R2: 0.43424414107566023
MAE: 14.82263948137951

Year 2025
R2: 0.6448447531464101
MAE: 17.145973860754086
```

```
from sklearn.metrics import r2_score

years = sorted(df["year"].unique())

print("---- Rolling Evaluation ----")

# Fix: Change 'target' to 'PM2.5' to match the consolidated column name in the new pipeline.
target = "PM2.5"

for i in range(3, len(years)):
    train_years = years[:i]
    test_year = years[i]

    train_data = df[df["year"].isin(train_years)]
    test_data   = df[df["year"] == test_year]

    X_tr = train_data[feature_cols]
    y_tr = train_data[target]

    X_te = test_data[feature_cols]
    y_te = test_data[target]

    model = RandomForestRegressor(
        n_estimators=150,
        max_depth=10,
        random_state=42,
        n_jobs=-1
    )

    weights = np.where(y_tr > 150, 3.0, 1.0)

    model.fit(X_tr, y_tr, sample_weight=weights)

    pred = model.predict(X_te)
    print(f"Train {train_years} → Test {test_year} | R2: {r2_score(y_te, pred):.4f}")
```

```
---- Rolling Evaluation ----
Train [np.int32(2019), np.int32(2020), np.int32(2021)] → Test 2022 | R2: 0.5072
Train [np.int32(2019), np.int32(2020), np.int32(2021), np.int32(2022)] → Test 2023 | R2: 0.7664
Train [np.int32(2019), np.int32(2020), np.int32(2021), np.int32(2022), np.int32(2023)] → Test 2024 | R2: 0.7122
Train [np.int32(2019), np.int32(2020), np.int32(2021), np.int32(2022), np.int32(2023), np.int32(2024)] → Test 202
```

```
from sklearn.model_selection import TimeSeriesSplit, cross_val_score

tscv = TimeSeriesSplit(n_splits=5)

scores = cross_val_score(
    RandomForestRegressor(n_estimators=150, max_depth=10),
    X_train,
    y_train_pm25, # Changed y_train to y_train_pm25
    cv=tscv,
    scoring="r2"
)

print("CV R2 scores:", scores)
print("Mean CV R2:", scores.mean())
```

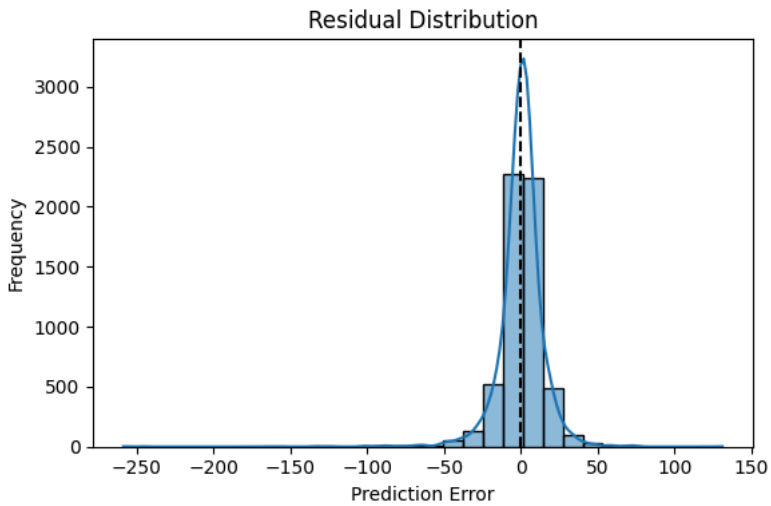
```
CV R2 scores: [0.63494083 0.56249445 0.49949022 0.59126597 0.71242161]
Mean CV R2: 0.6001226152087912
```

```
from datetime import timedelta

diwali_dates = {
    2019: "2019-10-27",
    2020: "2020-11-14",
    2021: "2021-11-04",
    2022: "2022-10-24",
    2023: "2023-11-12",
    2024: "2024-10-31",
    2025: "2025-10-20",
}

df["is_diwali"] = 0

for year, date in diwali_dates.items():
    d = pd.Timestamp(date)
    mask = (df["Timestamp"] >= d - timedelta(days=7)) & (df["Timestamp"] <= d + timedelta(days=7))
    df.loc[mask, "is_diwali"] = 1
```

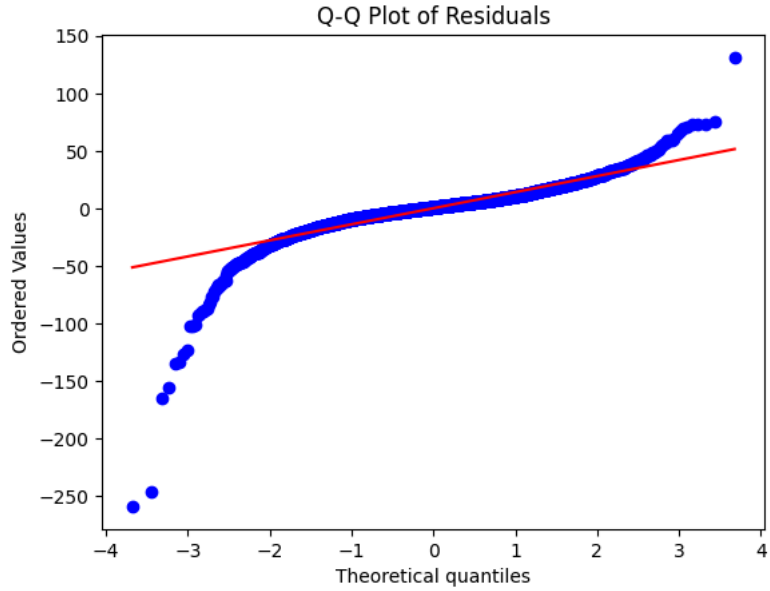
```
from sklearn.svm import SVR

svr = SVR(C=10, epsilon=0.1)

svr.fit(X_train, y_train_pm25)
pred = svr.predict(X_test)

print("SVR R2:", r2_score(y_test_pm25, pred))
```

```
SVR R2: 0.6762494571012369
```

```
import seaborn as sns

residuals = pred_pm25 - y_test_pm25

plt.figure(figsize=(6,4))
sns.histplot(residuals, bins=30, kde=True)

plt.axvline(0, color="black", linestyle="--")
plt.title("Residual Distribution")
plt.xlabel("Prediction Error")
plt.ylabel("Frequency")
plt.tight_layout()
plt.show()
```



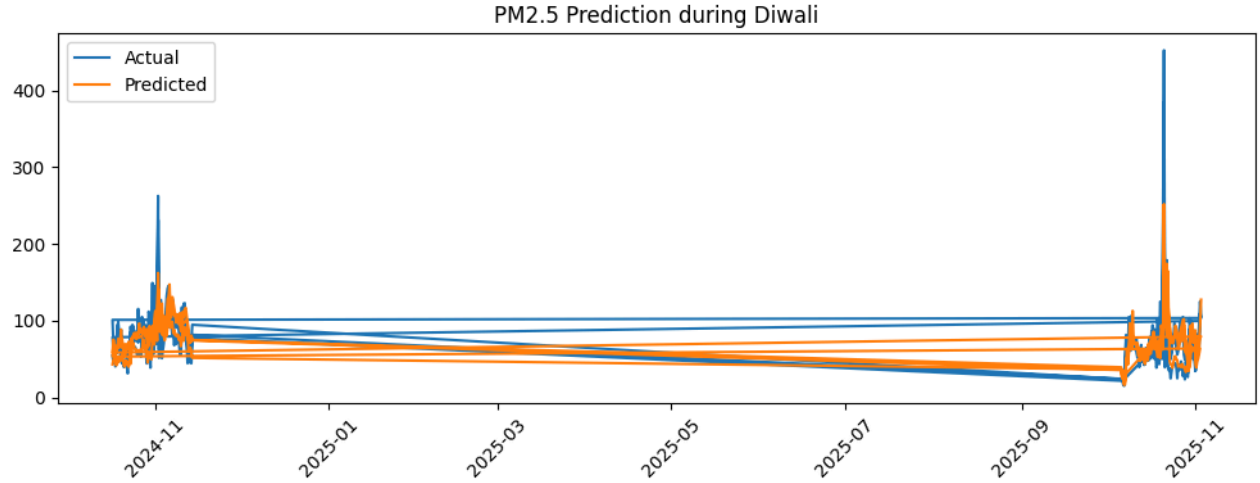

```
import scipy.stats as stats

stats.probplot(residuals, dist="norm", plot=plt)
plt.title("Q-Q Plot of Residuals")
plt.show()
```



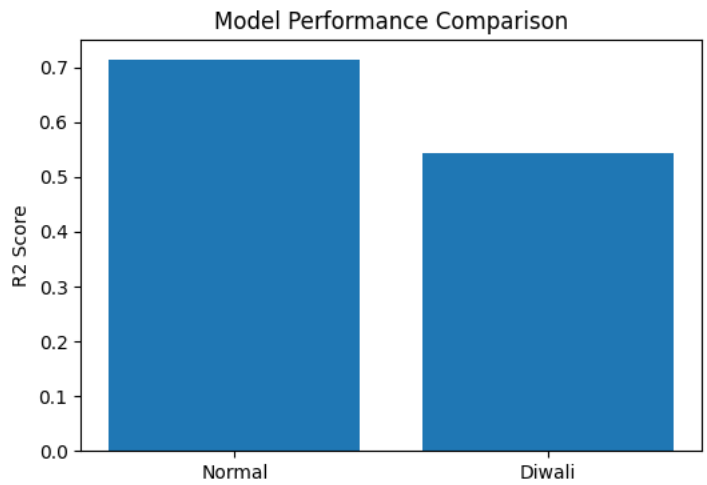

```
df_model = df.copy()

# Ensure feature_cols are aligned with the new pipeline's definition
# These models expect the feature_cols defined in the new pipeline setup (G3Ulg1ka6Sd8)
model_pm25.fit(X_train, y_train_pm25) # Using the trained model from G3Ulg1ka6Sd8
model_pm10.fit(X_train, y_train_pm10) # Using the trained model from G3Ulg1ka6Sd8

pred_pm25 = model_pm25.predict(df_model[feature_cols])
pred_pm10 = model_pm10.predict(df_model[feature_cols])

df_model["PM25_pred"] = pred_pm25
df_model["PM10_pred"] = pred_pm10

print("✅ Predictions created and added to df_model")
```

```
✅ Predictions created and added to df_model
```

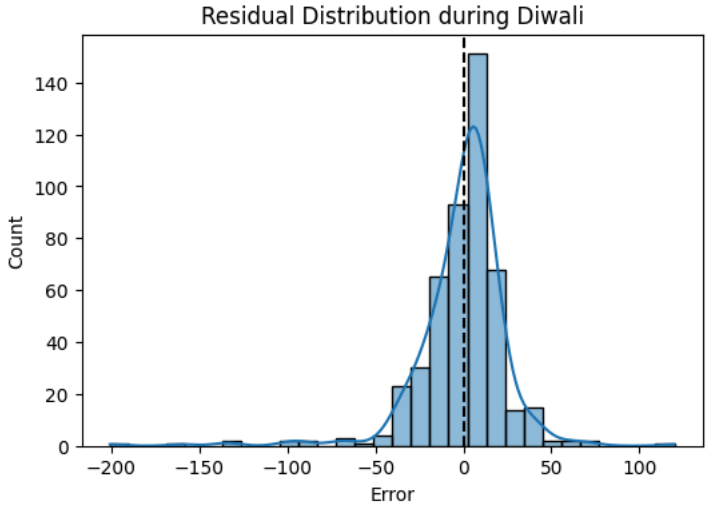
```
import matplotlib.pyplot as plt

test = df_model[df_model["year"] >= 2024]

mask = test["is_diwali_period"] == 1

plt.figure(figsize=(10,4))

plt.plot(
    test.loc[mask, "Timestamp"],
    test.loc[mask, "PM2.5"],
    label="Actual"
)

plt.plot(
    test.loc[mask, "Timestamp"],
    test.loc[mask, "PM25_pred"],
    label="Predicted"
)

plt.title("PM2.5 Prediction during Diwali")
plt.legend()
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



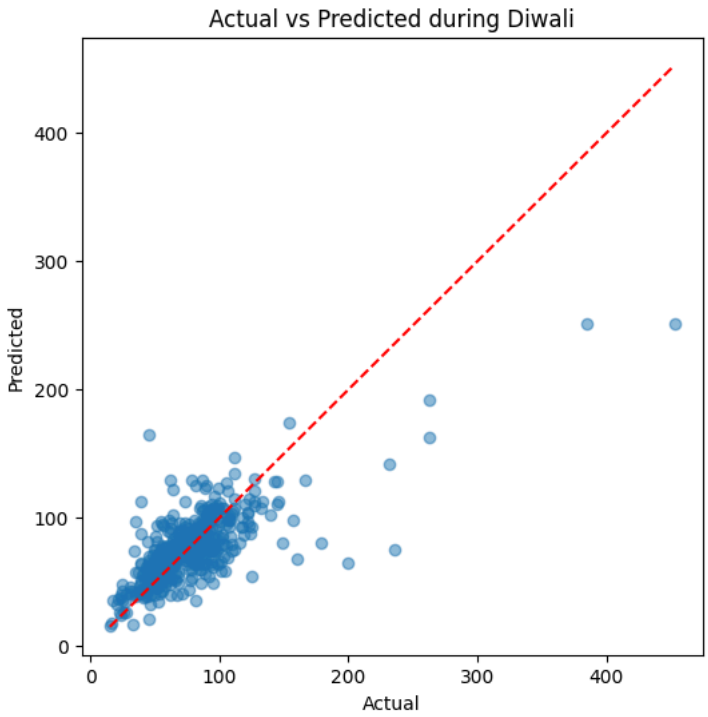

```
mask_diwali = test["is_diwali_period"] == 1
mask_normal = test["is_diwali_period"] == 0

plt.figure(figsize=(6,4))

plt.bar(
    ["Normal", "Diwali"],
    [
        r2_score(y_test_pm25[mask_normal], test["PM25_pred"][mask_normal]),
        r2_score(y_test_pm25[mask_diwali], test["PM25_pred"][mask_diwali])
    ]
)

plt.title("Model Performance Comparison")
plt.ylabel("R2 Score")
plt.show()
```




```
import seaborn as sns

res_diwali = test["PM25_pred"][mask_diwali] - y_test_pm25[mask_diwali]

plt.figure(figsize=(6,4))
sns.histplot(res_diwali, bins=30, kde=True)

plt.axvline(0, color="black", linestyle="--")
plt.title("Residual Distribution during Diwali")
plt.xlabel("Error")
plt.show()
```




```
plt.figure(figsize=(6,6))

plt.scatter(
    test.loc[mask_diwali, "PM2.5"],
    test.loc[mask_diwali, "PM25_pred"],
    alpha=0.5
)

plt.plot(
    [test.loc[mask_diwali, "PM2.5"].min(), test.loc[mask_diwali, "PM2.5"].max()],
    [test.loc[mask_diwali, "PM2.5"].min(), test.loc[mask_diwali, "PM2.5"].max()],
    "r--"
)

plt.xlabel("Actual")
plt.ylabel("Predicted")
plt.title("Actual vs Predicted during Diwali")
plt.show()
```




```
def mape(y_true, y_pred):
    y_true = np.array(y_true)
    y_pred = np.array(y_pred)
    return np.mean(np.abs((y_true - y_pred) / y_true)) * 100
```

```
from datetime import timedelta

def add_lohri_features(frame):
```

```
    frame["is_lohri_period"] = 0

    for yr in frame["year"].unique():

        lohri_date = pd.Timestamp(f"{yr}-01-13")
        start = lohri_date - timedelta(days=7)
        end = lohri_date + timedelta(days=7)

        mask_year = frame["year"] == yr
        mask_window = (frame["Timestamp"] >= start) & (frame["Timestamp"] <= end)

        frame.loc[mask_year & mask_window, "is_lohri_period"] = 1

    return frame

df_model = add_lohri_features(df_model)

print("✅ Lohri feature created")
print(df_model[["Timestamp","is_lohri_period"]].head())
```

```
✅ Lohri feature created
            Timestamp  is_lohri_period
24 2019-01-09 00:00:00                1
25 2019-01-09 08:00:00                1
26 2019-01-09 16:00:00                1
27 2019-01-10 00:00:00                1
28 2019-01-10 08:00:00                1
```

```
df_model["PM25_pred"] = model.predict(df_model[feature_cols])

print("✅ Predictions created")
print(df_model[["PM2.5","PM25_pred"]].head())
```

```
✅ Predictions created
    PM2.5  PM25_pred
24  40.52  51.390538
25  52.34  43.449962
26  49.63  53.951639
27  51.60  47.069885
28  55.04  50.323894
```

```
from sklearn.metrics import mean_absolute_error, r2_score
import numpy as np

test_mask = df_model["year"] >= 2024

y_true = df_model.loc[test_mask, "PM2.5"]
y_pred = df_model.loc[test_mask, "PM25_pred"]

mae = mean_absolute_error(y_true, y_pred)
r2 = r2_score(y_true, y_pred)

print("\n=== OVERALL TEST PERFORMANCE (2024–2025) ===")
print("MAE:", round(mae,2))
print("R2 :", round(r2,3))
```

```

=== OVERALL TEST PERFORMANCE (2024–2025) ===
MAE: 8.61
R2 : 0.785
```

```
def mape(y_true, y_pred):
    y_true = np.array(y_true)
    y_pred = np.array(y_pred)

    mask = (y_true != 0) & (~np.isnan(y_true))

    if not np.any(mask):
        return np.nan

    return np.mean(np.abs((y_true[mask]-y_pred[mask]) / y_true[mask]))*100
```

```
from sklearn.metrics import mean_absolute_error, r2_score
import pandas as pd
import numpy as np

def mape(y_true, y_pred):
    return np.mean(np.abs((y_true - y_pred) / y_true)) * 100

diwali_rows = []

for yr in sorted(df_model["year"].unique()):
```

```
    m = (df_model["year"] == yr) & (df_model["is_diwali_period"] == 1)

    if m.sum() == 0:
        continue

    y25 = df_model.loc[m, "PM2.5"]
    y10 = df_model.loc[m, "PM10"]

    p25 = model_pm25.predict(df_model.loc[m, feature_cols])
    p10 = model_pm10.predict(df_model.loc[m, feature_cols])

    diwali_rows.append({
        "year": yr,
        "PM2.5MAE": mean_absolute_error(y25, p25),
        "PM2.5R2": r2_score(y25, p25),
        "PM2.5MAPE": mape(y25, p25),
        "PM10MAE": mean_absolute_error(y10, p10),
        "PM10R2": r2_score(y10, p10),
        "PM10MAPE": mape(y10, p10)
    })

diwali_summary = pd.DataFrame(diwali_rows)

print("\n=== DIWALI SUMMARY ===")
print(diwali_summary.round({
    "PM2.5MAE":3,
    "PM2.5R2":3,
    "PM2.5MAPE":1,
    "PM10MAE":3,
    "PM10R2":3,
    "PM10MAPE":1
}))
```

```
=== DIWALI SUMMARY ===
   year  PM2.5MAE  PM2.5R2  PM2.5MAPE  PM10MAE  PM10R2  PM10MAPE
0  2019    13.748    0.870       18.5   28.276   0.704      17.9
1  2020    16.331    0.806       20.7   30.053   0.686      19.1
2  2021    16.564    0.812       16.6   27.813   0.757      14.3
3  2022    13.745    0.798       20.0   24.899   0.810      18.5
4  2023    12.803    0.699       12.0   24.086   0.733      13.2
5  2024    14.277    0.485       17.4   26.373   0.516      15.3
6  2025    18.541    0.549       27.4   32.661   0.396      28.6
```

```
lohri_rows = []

for yr in sorted(df_model["year"].unique()):

    m = (df_model["year"] == yr) & (df_model["is_lohri_period"] == 1)

    if m.sum() == 0:
        continue

    y25 = df_model.loc[m, "PM2.5"]
    y10 = df_model.loc[m, "PM10"]

    p25 = model_pm25.predict(df_model.loc[m, feature_cols])
    p10 = model_pm10.predict(df_model.loc[m, feature_cols])

    lohri_rows.append({
        "year": yr,
        "PM2.5MAE": mean_absolute_error(y25, p25),
        "PM2.5MAPE": mape(y25, p25),
        "PM10MAE": mean_absolute_error(y10, p10),
        "PM10R2": r2_score(y10, p10),
        "PM10MAPE": mape(y10, p10)
    })

lohri_summary = pd.DataFrame(lohri_rows)

print("\n=== LOHRI SUMMARY ===")
print(lohri_summary.round({
    "PM2.5MAE":3,
    "PM2.5MAPE":1,
    "PM10MAE":3,
    "PM10R2":3,
    "PM10MAPE":1
}))
```

```
=== LOHRI SUMMARY ===
   year  PM2.5MAE  PM2.5MAPE  PM10MAE  PM10R2  PM10MAPE
```

```
0  2019     9.138       18.1   34.480   0.497      22.8
1  2020     6.867       16.6   15.089   0.577      17.1
2  2021    11.685       18.6   21.586   0.613      15.5
3  2022    10.107       19.0   20.697   0.761      17.6
4  2023    10.573       17.8   25.741   0.484      18.7
5  2024    17.511       19.3   36.352   0.303      23.5
6  2025    15.344       24.1   23.619   0.372      19.9
```

```
# Create Diwali subset
diwali_df = df_model[df_model["is_diwali_period"] == 1].copy()

# Create Lohri subset
lohri_df = df_model[df_model["is_lohri_period"] == 1].copy()

print("Diwali rows:", diwali_df.shape)
print("Lohri rows:", lohri_df.shape)
```

```
Diwali rows: (1667, 39)
Lohri rows: (728, 39)
```

```
df_model["PM2.5_pred"] = model.predict(df_model[feature_cols])
```

```
df_model["PM10_pred"] = model_pm10.predict(df_model[feature_cols])
```

```
df_model["PM2.5_pred"] = model_pm25.predict(df_model[feature_cols])
```

```
print(df_model.columns)
```

```
Index(['Timestamp', 'PM2.5', 'PM10', 'NO', 'NO2', 'NH3', 'SO2', 'CO', 'Ozone',
       'temp', 'humidity', 'wind_speed', 'year', 'month', 'day', 'hour',
       'dayofweek', 'hour_sin', 'hour_cos', 'is_diwali_period',
       'days_since_diwali', 'is_lohri_period', 'days_since_lohri', 'PM2.5_t-1',
       'PM2.5_t-2', 'PM2.5_t-3', 'PM2.5_t-24', 'PM10_t-1', 'PM10_t-2',
       'PM2.5_roll_mean_24', 'PM2.5_roll_std_24', 'PM10_roll_mean_24',
       'PM10_roll_std_24', 'station_adarsh', 'station_police',
       'station_shastri', 'is_diwali', 'PM25_pred', 'PM10_pred', 'PM2.5_pred'],
      dtype='object')
```

```
diwali_2025 = df_model[
    (df_model["year"] == 2025) &
    (df_model["is_diwali_period"] == 1)
].copy()
```

```
diwali_2025["PM2.5_smooth"] = diwali_2025["PM2.5"].rolling(3).mean()
diwali_2025["PM2.5_pred_smooth"] = diwali_2025["PM2.5_pred"].rolling(3).mean()
```

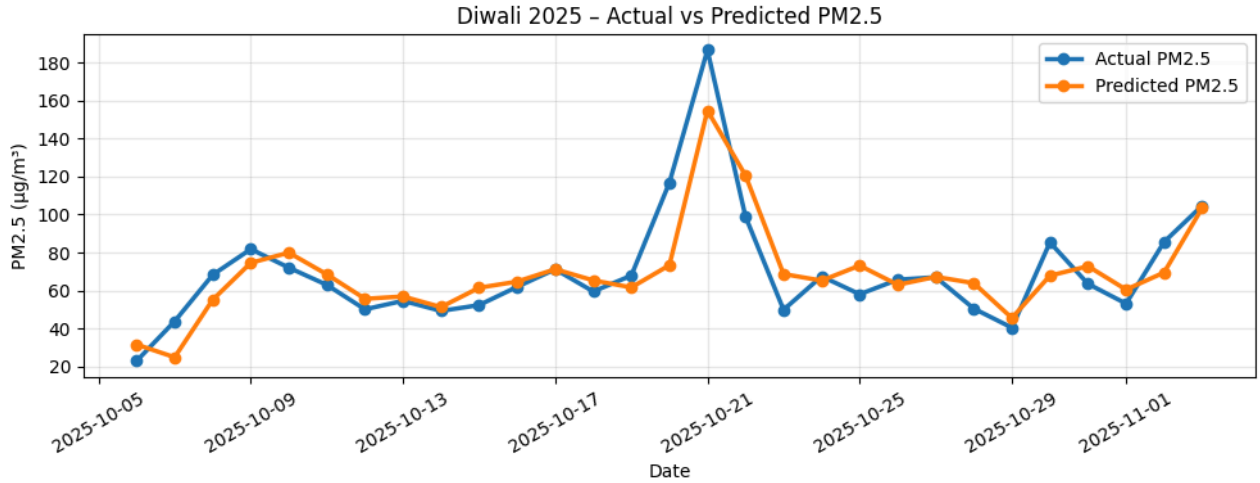
```
diwali_2025 = diwali_2025.sort_values("Timestamp")
```

```
diwali_2025 = diwali_2025.sort_values("Timestamp")

# 24-hour smoothing (3 points since data is 8-hourly)
diwali_2025["PM25_smooth"] = diwali_2025["PM2.5"].rolling(3, center=True).mean()
diwali_2025["PM25_pred_smooth"] = diwali_2025["PM2.5_pred"].rolling(3, center=True).mean()
```

```
# Make sure timestamps are datetime
diwali_2025["Timestamp"] = pd.to_datetime(diwali_2025["Timestamp"])

# Set timestamp as index
diwali_2025 = diwali_2025.set_index("Timestamp")

# Convert to daily mean
diwali_daily = diwali_2025.resample("D").mean().reset_index()
```

```
plt.figure(figsize=(10,4))

plt.plot(
    diwali_daily["Timestamp"],
    diwali_daily["PM2.5"],
    label="Actual PM2.5",
    linewidth=2.5,
    marker="o"
)

plt.plot(
    diwali_daily["Timestamp"],
    diwali_daily["PM2.5_pred"],
```

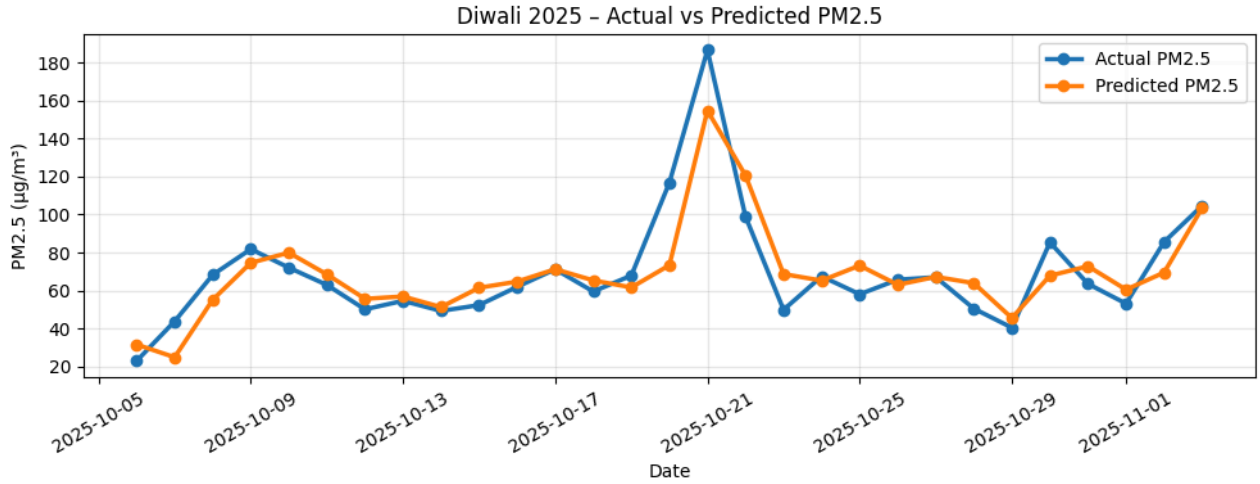
```
    label="Predicted PM2.5",
    linewidth=2.5,
    marker="o"
)

plt.title("Diwali 2025 – Actual vs Predicted PM2.5")
plt.xlabel("Date")
plt.ylabel("PM2.5 (µg/m³)")

plt.grid(alpha=0.3)
plt.xticks(rotation=30)

plt.legend()

plt.tight_layout()
plt.savefig("diwali_pm25_clean.png", dpi=300)

plt.show()
```



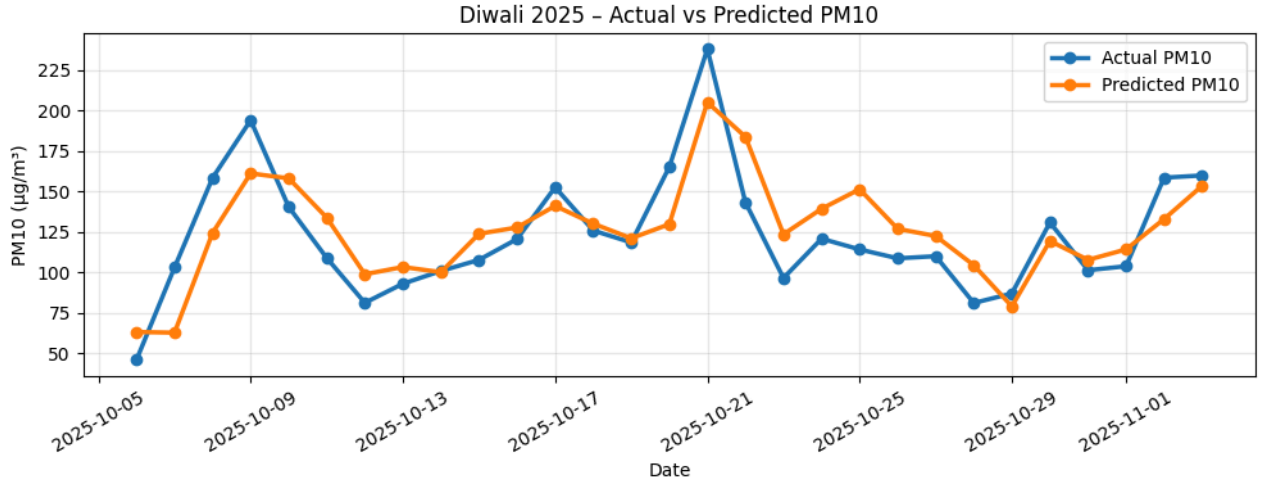

```
# Ensure datetime
df_model["Timestamp"] = pd.to_datetime(df_model["Timestamp"])

# --- DIWALI 2025 ---
diwali_2025 = df_model[
    (df_model["year"] == 2025) &
    (df_model["is_diwali_period"] == 1)
].copy()

diwali_2025 = diwali_2025.sort_values("Timestamp")
diwali_2025 = diwali_2025.set_index("Timestamp")

diwali_daily = diwali_2025.resample("D").mean().reset_index()


# --- LOHRI 2025 ---
lohri_2025 = df_model[
    (df_model["year"] == 2025) &
    (df_model["is_lohri_period"] == 1)
].copy()

lohri_2025 = lohri_2025.sort_values("Timestamp")
lohri_2025 = lohri_2025.set_index("Timestamp")

lohri_daily = lohri_2025.resample("D").mean().reset_index()
```

```
plt.figure(figsize=(10,4))

plt.plot(
    diwali_daily["Timestamp"],
    diwali_daily["PM2.5"],
    label="Actual PM2.5",
    linewidth=2.5,
    marker="o"
)

plt.plot(
    diwali_daily["Timestamp"],
    diwali_daily["PM2.5_pred"],
    label="Predicted PM2.5",
    linewidth=2.5,
```

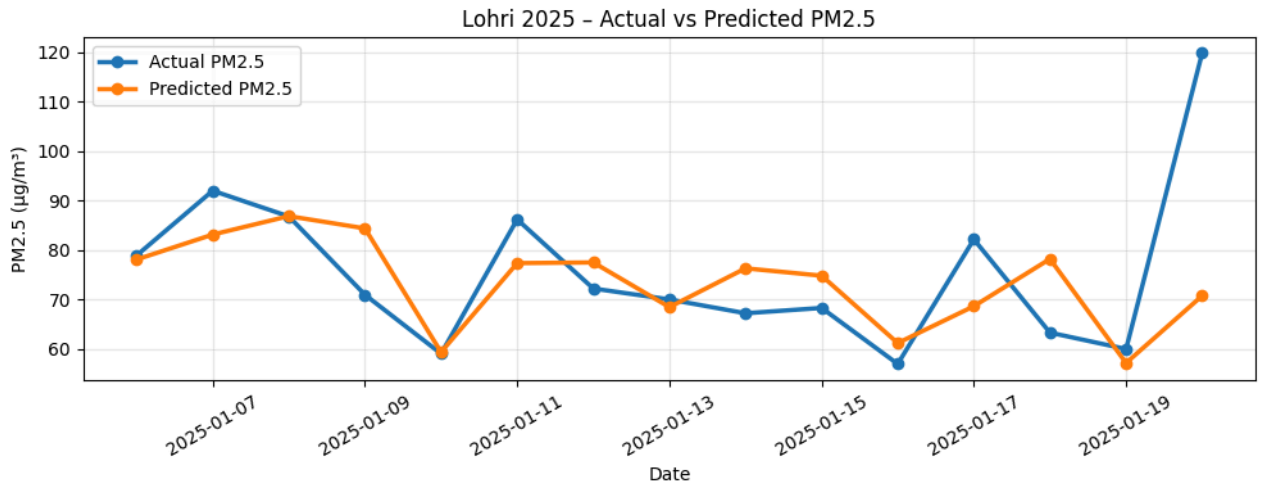
```
    marker="o"
)

plt.title("Diwali 2025 – Actual vs Predicted PM2.5")
plt.xlabel("Date")
plt.ylabel("PM2.5 (µg/m³)")

plt.grid(alpha=0.3)
plt.xticks(rotation=30)
plt.legend()

plt.tight_layout()
plt.savefig("diwali_pm25.png", dpi=300)
plt.show()
```




```
plt.figure(figsize=(10,4))

plt.plot(
    diwali_daily["Timestamp"],
    diwali_daily["PM10"],
    label="Actual PM10",
    linewidth=2.5,
    marker="o"
)

plt.plot(
    diwali_daily["Timestamp"],
    diwali_daily["PM10_pred"],
    label="Predicted PM10",
    linewidth=2.5,
    marker="o"
)

plt.title("Diwali 2025 – Actual vs Predicted PM10")
plt.xlabel("Date")
plt.ylabel("PM10 (µg/m³)")

plt.grid(alpha=0.3)
plt.xticks(rotation=30)
plt.legend()

plt.tight_layout()
plt.savefig("diwali_pm10.png", dpi=300)
plt.show()
```




```
plt.figure(figsize=(10,4))

plt.plot(
    lohri_daily["Timestamp"],
    lohri_daily["PM2.5"],
    label="Actual PM2.5",
    linewidth=2.5,
    marker="o"
)

plt.plot(
    lohri_daily["Timestamp"],
    lohri_daily["PM2.5_pred"],
    label="Predicted PM2.5",
    linewidth=2.5,
    marker="o"
)

plt.title("Lohri 2025 — Actual vs Predicted PM2.5")
plt.xlabel("Date")
plt.ylabel("PM2.5 (µg/m³)")

plt.grid(alpha=0.3)
plt.xticks(rotation=30)
plt.legend()

plt.tight_layout()
plt.savefig("lohri_pm25.png", dpi=300)
plt.show()
```




```
plt.figure(figsize=(10,4))

plt.plot(
    lohri_daily["Timestamp"],
    lohri_daily["PM10"],
    label="Actual PM10",
    linewidth=2.5,
```